Week 1 questions — exam-style answers

These are **full answers**, written the same way as THEORY-NOTES.md.
Read the question. Cover this page. Write it yourself. Then check.

Same questions: 01-Week1-Questions.pdf and 01-Week1-Questions-v2.pdf.

Easy — Concept recall

1. What is the difference between a grayscale image and a color image in terms of matrix representation?

A computer does not see a photo the way you do. It only sees numbers. Each tiny square of the image is a **pixel**.

A **grayscale** image stores **one number per pixel**, usually from 0 to 255. So the whole image is a **2D matrix**: rows × columns, that is height × width. Shape looks like `(height, width)`, for example `(5472, 3648)`.

A **color** pixel needs **three** numbers: Red, Green and Blue. So a color image is a **3D matrix**: height × width × **3 channels**. Shape looks like `(height, width, 3)`.

Paper line (from notes): *A grayscale image is a 2D matrix of intensities 0–255. A color image is a 3D matrix with 3 channels.*

---

2. In a grayscale image, what do pixel values 0 and 255 represent?

In grayscale, one number per pixel tells brightness.

- **0** means **black** (no light).
- **255** means **white** (full brightness).
- A number in the middle, like **128**, is **gray**.

So 0 and 255 are the two ends of the brightness scale.

---

3. How many channels does an RGB image have?

An RGB image has **3 channels**: Red, Green and Blue.
Every color pixel is three numbers mixed together. That is why the matrix is height × width × 3.

---

4. What is the coordinate system origin in computer vision images?

In maths, the origin is often at the **bottom-left**, and +y goes **up**.

In images, origin (0, 0) is at the top-left.

- `x` grows to the **right**
- `y` grows **down**

"Top-left" only tells you **where (0,0) sits**. The x-axis still goes right. The only change is the y-axis: in images it goes down.

Why top-left? Screens and image files are stored **row by row starting from the first row at the top**.

---

5. How do you access the pixel at position (x=10, y=5) in OpenCV?

You use `image[y, x]` — **row first, then column**.

So the pixel at x=10, y=5 is `image[5, 10]`. That means "5 down, 10 across", not the other way.

If you write `image[10, 5]` you are reading the wrong place.

---

6. Which color order does OpenCV use by default?

OpenCV stores color as BGR (Blue, Green, Red), not RGB.

Most websites and **Matplotlib** use **RGB**. Same three numbers, different order. If you display an OpenCV image in Matplotlib without converting, **red and blue look swapped**.

---

7. What does HSV stand for?

HSV stands for **Hue, Saturation, Value**.

It is a color model that separates **color** from **brightness**. That is why it is used for color detection.

---

8. In HSV, what does the Hue component represent?

Hue is **which color family** the pixel belongs to — red, yellow, green, blue.

Think of it as the **angle on a color wheel**: about 0° red, 120° green, 240° blue.

Hue is the actual color name. When light changes, Hue usually stays almost the same. Value (brightness) is what changes.

---

9. What does Saturation in HSV indicate?

Saturation is **how pure / washed** the color is.

- **0%** = gray (no color, only brightness)
- **100%** = vivid, strong color

A pale pink and a bright red can share the same Hue. Saturation is what makes one look washed and the other look strong.

---

10. What does Value in HSV indicate?

Value is **how bright** the color is (intensity).

- **0%** = black
- **100%** = the brightest version of that hue

When a yellow banana moves from sunlight into shade, Hue stays yellow. **Value** is the part that drops.

---

11. Define a kernel in image processing.

A **kernel** (also called a filter, mask, or convolution matrix) is a **small matrix of numbers**, usually 3×3 or 5×5, that slides over an image.

At each position, it combines the center pixel and its neighboring pixels to calculate a **new** pixel value. You place it on a pixel and its neighbors, multiply matching cells, and **add them up**. That sum becomes the new pixel.

Different kernels do different jobs: blur, sharpen, or find edges.

Paper line: *A kernel is a small matrix that slides over an image. At each position we multiply overlapping values and sum them to get a new pixel. Different kernels produce blur, sharpen or edges.*

---

12. What is the purpose of convolution in image filtering?

Convolution is the name of that sliding process. A small kernel moves across the image, multiplying and summing pixel values to produce a new output.

Its purpose is to **extract features** such as edges, textures, or smoothness.

- Blur kernel → smooths the image
- Sharpen kernel → makes edges clearer

- Edge detection kernel → finds boundaries

The output is a **feature map** — a new image that highlights whatever the kernel cares about.

Paper line: *Convolution is a sliding-window operation. A kernel moves across the image, computes a weighted sum of local pixels, and produces a feature map. It is the foundation of both classical filtering and CNNs.*

---

13. What does a histogram represent in image processing?

A **histogram** counts how many pixels have each intensity from 0 to 255.

- X-axis: intensity (dark → bright)
- Y-axis: how many pixels

It does **not** tell you what objects are in the photo. It only tells you about **brightness and contrast**. That is still enough to decide "should I enhance this?"

Paper line: *A histogram shows the distribution of pixel intensities. It helps judge brightness and contrast. A left-shifted histogram is underexposed; a right-shifted one is overexposed; a narrow histogram means low contrast.*

---

14. What is thresholding used for?

Thresholding converts a grayscale image into a black-and-white (binary) image by choosing a cutoff value called the threshold **T**.

- If pixel intensity > T → set to white (255)
- If pixel intensity ≤ T → set to black (0)

This removes gray shades and leaves only black and white, so objects are easier to separate from the background.

It is used in **document scanning**, **object detection**, and **medical image analysis**.

---

15. Name two basic morphological operations.

The two basic operations are **dilation** and **erosion**.

Dilation **grows** the white object. Erosion **shrinks** it.

Opening and **closing** are built from these two. Opening is erosion then dilation. Closing is dilation then erosion. Remember the order.

Medium — Application and explanation

1. Explain why HSV is often preferred over RGB for color detection tasks.

RGB creates colors by mixing Red, Green and Blue light. It is excellent for **displaying** images, because screens emit those three lights. It is **not reliable for color detection**, because the RGB values of the **same object change with lighting**.

Example: a yellow banana in bright sunlight might be `(255, 255, 50)`. The same banana in shade might be `(120, 120, 20)`. Both look yellow to your eyes, but the numbers are very different. If you search for a fixed RGB value, you miss many yellow objects.

HSV separates color from brightness.

- Banana in sun: Hue = Yellow, Saturation high, Value high
- Same banana in shade: Hue still Yellow, Saturation almost the same, **only Value dropped**

Since **Hue remains almost the same**, the computer can still recognise yellow when lighting changes.

Exam definition (from notes): *HSV (Hue, Saturation, Value) is a color model that separates color from brightness. Hue represents the color, Saturation represents the purity of the color, and Value represents the brightness. HSV is preferred for color detection than RGB because changes in lighting mainly affect the Value, while the Hue remains relatively stable.*

---

2. Give an example of a pixel value in RGB that represents pure red.

Pure red in RGB is `[255, 0, 0]` — red full, green off, blue off.

(In OpenCV the same pixel is stored as BGR `[0, 0, 255]`. If the question says RGB, write `[255, 0, 0]`.)

---

3. What happens when you apply a blur kernel to an image?

A blur kernel (box / average, or Gaussian) **mixes a pixel with its neighbors**.

The center pixel is replaced by a weighted average of the window. Sharp noise goes down. The image looks **smoother**. Fine detail is reduced.

Gaussian blur is the same idea, but the **center counts more** than far neighbors, so the blur looks more natural.

A **larger** kernel (7×7 instead of 3×3) looks at a wider neighborhood: stronger blur, slower, less tiny detail.

---

4. How does an edge detection kernel work?

An **edge** is a place where brightness **jumps** — the outline of an object.

An edge kernel (for example **Sobel**) looks at neighbors and gives a **high score** where they are very different.

- **Sobel X** finds **vertical** edges (left vs right change)
- **Sobel Y** finds **horizontal** edges (up vs down change)

Together they give an edge map. Sobel is simple but a bit noisy.

Canny is an advanced edge detector. It also finds edges, but it removes noise and keeps only the most important, clean, thin edges. Used in lane detection, document outlines, medical boundaries.

Paper line: *Edge detection finds pixels where intensity changes sharply. Sobel uses gradient kernels for horizontal and vertical edges. Canny combines smoothing, gradient, non-max suppression and double thresholding, so edges are cleaner.*

---

5. Compare global thresholding and adaptive thresholding.

Thresholding converts grayscale to binary using a cutoff T.

Global thresholding uses **one T for the entire image**. Example: whole image → T = 127. It works when lighting is **uniform.** If one part is bright and another is dark, one threshold cannot separate the object everywhere.

Adaptive thresholding calculates a **different T for each small region**. Bright area → T = 150, dark area → T = 80. This works when lighting is **uneven**, because each region gets its own suitable threshold.

Otsu (if they also ask): you do not choose T. The computer automatically finds the best T that separates foreground and background. Best when the histogram has two clear peaks, such as black text on a white page.

Paper line: *Thresholding converts grayscale to binary. Global uses one cutoff and fails under uneven light. Adaptive uses local cutoffs. Otsu automatically chooses T by minimizing intra-class variance.*

---

6. What information can be derived from an image histogram about brightness and contrast?

A histogram shows how many pixels sit at each intensity 0–255. From the **shape** you can read:

- Peak on the **left** → the image is **dark** (underexposed)

- Peak on the **right** → the image is **bright** (overexposed)
- A **narrow** peak → **low contrast** (washed / foggy) — pixels are not using the full range
- **Spread** across 0–255 → **good contrast**

It does not tell you *what* is in the photo. It only tells brightness and contrast. That is enough to decide if you should enhance the image (for example with histogram equalization) before the next step.

---

7. How does dilation differ from erosion in morphological operations?

Morphological operations **change the shape of objects**, mainly on binary images. White = object, black = background. They use a small **structuring element** that slides over the image.

Dilation grows the object. It makes the white region **bigger**, closes small gaps, and **connects broken parts**. Example: connect broken letters.

Erosion shrinks the object. It makes the white region **smaller** and **removes tiny white dots** (noise). Example: thinning lines or removing stray dots.

So dilation is the marker. Erosion is the eraser.

---

8. Explain the difference between opening and closing operations.

Both are made from dilation and erosion. The **order** is the whole answer.

Opening = erosion then dilation. First remove small noise, then restore the main object. Think: **clean the object**. Used on a noisy scanned page.

Closing = dilation then erosion. First fill small holes or gaps, then restore the object's size. Think: **repair the object**. Used to fill holes and reconnect broken characters.

Paper line: *Opening = erosion then dilation (remove noise). Closing = dilation then erosion (fill holes).*

---

9. Why are structuring elements important in morphological operations?

A **structuring element** is the small shape (3×3 square, circle, or cross) that **slides over the image** and decides how each pixel should change based on its neighbors.

Without it, the computer would not know *how far* to grow or shrink. **Size** decides how strong the effect is. A bigger element is stronger: it can remove bigger noise, but it can also **eat thin letters** or merge nearby objects.

So the structuring element is what makes morphology a shape tool, not a color tool.

---

10. What is a convex hull in contour analysis?

After thresholding, a white blob has a boundary. That boundary curve is a **contour**.

A **convex hull** is like putting a **rubber band** around the shape. It is the **smallest convex polygon** that encloses the contour. There are **no dents**.

It simplifies the outline. It is useful when you care about the overall shape, not every small notch.

Paper line: *A convex hull is the smallest convex polygon enclosing the contour.*

---

11. How can bounding boxes be used in object localization?

From a contour you can compute a **bounding box**: the **smallest upright rectangle** around the object.

Localization means answering "**the object is here**". The box gives that place — used to mark a face, a fruit, a letter, or a tumour region — so the next step (classify, count, track) knows where to look.

It does **not** give the exact shape. For the exact shape you need the contour or a mask. The box is the simple "here" answer.

---

12. Why is the origin (0,0) at the top-left corner in image representation?

Images are stored **row by row starting from the first row at the top**, like reading a page. Screens also start drawing from the top-left.

That is why the image coordinate system is not the maths one:

- Maths: (0,0) bottom-left, +y up
- Images: (0,0) **top-left**, +y **down**
- In both, +x still goes **right**

So `image[y, x]` means row first.

---

13. How does Matplotlib's RGB order differ from OpenCV's BGR order?

OpenCV stores color as **BGR** (Blue, Green, Red).
Matplotlib (and most websites) use **RGB**.

Same three numbers, **different order**. If you display an OpenCV image in Matplotlib **without converting**, red and blue look swapped. A red apple can look blue.

Before you plot, convert BGR → RGB.

---

14. What is the role of kernels in deep learning CNNs compared to classical image processing?

The **sliding idea is the same**. A small kernel still multiplies neighbors and adds.

In **traditional image processing**, we **manually design** the kernel values for a specific task — blur, sharpen, Sobel edges.

In a **CNN**, we still use kernels, but **we do not choose their values ourselves**. The CNN **learns the best kernel values automatically from training data**. During training it adjusts the numbers to detect edges, textures, shapes, and later faces or cars.

That is why CNNs are more flexible than hand-made filters.

Paper line: *In CNNs, kernels are learned automatically to detect patterns.*

---

15. Explain how thresholding can be applied in medical imaging.

A medical scan is usually **grayscale**. The organ or tumour is often a different brightness from the background.

Thresholding turns that scan into **black and white**: the region of interest becomes white, the rest black. Then you can measure **size and shape**, or find a contour around the organ.

If lighting or contrast is uneven (common in real scans), **one global T** can miss part of the organ. Then use **adaptive** thresholding or **Otsu** if there are two clear groups in the histogram. After that, morphological opening/closing can clean small holes before the doctor (or the next algorithm) uses the mask.

Hard — Analytical and design

1. Derive the mathematical expression for 2D convolution.

Convolution is a sliding-window operation. Put kernel `K` on image `I`. At each position, multiply overlapping numbers and add. That sum is the new pixel. Then slide and repeat.

The 2D formula is:

[(I * K)(x, y) = \sum_m \sum_n I(x - m,\ y - n) \cdot K(m, n)]

`I` is the image, `K` is the kernel. Do not panic: it only says “look at neighbors, weight them with K, add”.

The output is a **feature map**. This same operation is the foundation of both classical filtering and CNNs.

---

2. Discuss the limitations of global thresholding in images with uneven illumination and propose solutions.

Limit: Global thresholding uses **one T for the whole image**. If the page is bright on the left and dark on the right, one cutoff cannot work. Text is lost on one side. This is the uneven-light problem from the notes.

Solutions (write two or three):

1. **Adaptive thresholding** — a different T for each small region, so each area follows its own light.
2. **Otsu** — if the histogram still has two clear peaks, let the computer pick T. (Otsu still uses one T, so it does not fully fix uneven light. Mention it as "automatic T", then prefer adaptive.)
3. **Histogram equalization first** — stretch contrast so dark and bright regions use more of 0–255, then threshold.
4. After the binary image: **opening / closing** to clean leftover specks and holes.

Paper line: *Global uses one cutoff and fails under uneven light. Adaptive uses local cutoffs.*

---

3. How can histogram equalization improve image contrast?

Suppose most pixels sit between 40 and 90. The photo looks dull because it is not using the full 0–255 range.

Histogram equalization remaps intensities so they use the full 0–255 range. Darks get darker, brights get brighter. Contrast improves. Details become easier to see.

It is often done **before** thresholding or feature detection.

Limit: it can also **boost noise**.

Paper line: *Histogram equalization remaps intensities so they use the full 0–255 range, which improves contrast. It is useful as a preprocessing step but can amplify noise.*

---

4. Design a pipeline to detect ripe fruit using HSV color space.

RGB is bad here: a ripe yellow/red fruit in sun and in shade has different RGB numbers. HSV keeps **Hue** more stable.

A simple pipeline:

1. Read the image. Convert **BGR to HSV** (OpenCV is BGR).
2. Choose a **Hue range** for ripe color (yellow or red). Also keep Saturation and Value in a useful band so you ignore gray dirt and pure black.
3. Use that range as a threshold (`inRange`) to get a **binary mask**: white = ripe pixels.

4. **Clean the mask** with morphology: **opening** to remove specks, **closing** to fill holes.
5. **Find contours**. Keep only large blobs (ignore tiny noise).
6. Draw a **bounding box** (or the contour) around each remaining blob. That box **localises** the fruit.

This is HSV segmentation + contour analysis, the same pairing as the last hard question.

---

5. Explain how contour approximation (Douglas–Peucker) can be used for shape classification.

A **contour** is a curve joining continuous boundary points of the same intensity — the outline of an object.

A real contour is **wiggly** (hundreds of points). **Douglas–Peucker** simplifies it to a few corners (polygon approximation).

Then you **count the vertices**:

- 3 corners → triangle
- 4 corners → rectangle / square
- many corners → circle (or a smooth blob)

Paper line: *Vertex count after approximation can classify shapes.*

You can also use area and perimeter with this, but the exam point is: simplify, then count corners.

---

6. How can morphological operations be combined to clean noisy scanned documents?

A scan is often gray, with ink specks and broken letters.

1. Convert to gray. Use **Otsu** (or adaptive if the page is uneven) to get black text on white (or the opposite).
2. **Opening** (erosion then dilation) with a **small** structuring element — removes tiny ink specks, then restores the letters.
3. **Closing** (dilation then erosion) — fills small holes and reconnects broken characters.
4. If letters are still thin, a light **dilation** can make them more readable.

Do not use a **huge** structuring element: it will eat thin letters or glue two words together.

This is a standard document-processing use of morphology from the notes.

---

7. Propose a method to separate overlapping objects using thresholding, morphological operations, and contour analysis.

When two white blobs **touch**, `findContours` may treat them as **one** object.

A method:

1. **Threshold** the gray image (adaptive if light is uneven) so objects are white.
2. **Erode** (or open) with a structuring element a bit smaller than the objects. Each blob **shrinks**. The thin bridge between two objects **breaks**. Now they are separate.
3. **Find contours**. Each contour is one object. Compute a **bounding box** or centroid for each.
4. Optional: **dilate** each object again so it returns near its original size, or keep the eroded contour only for counting.

This combination: threshold → morphology (split) → contours (one outline each).

---

8. Compare the effectiveness of RGB vs HSV in detecting traffic lights under varying lighting conditions.

A traffic light must be found in **sun, evening, and night lamps**. That is an illumination change.

RGB: red / yellow / green are mixtures of three lights. The same red lamp has very different RGB numbers in daylight and at night. A fixed RGB range **misses** the light.

HSV: Hue of red, yellow and green stays **more stable**. Value can jump when the lamp is bright; you can **ignore Value** (or keep a wide band) and search Hue.

So **HSV is more effective** under varying light, for the same reason as the banana example.

Limit of HSV: red Hue wraps around 0° and 180° in OpenCV, so you may need **two** red ranges. Still better than RGB for this task.

---

9. How does kernel size affect the output of convolution operations?

The kernel looks at a **neighborhood**. Size is how wide that neighborhood is.

A **small kernel (3×3)** looks at a tiny area. It keeps **fine detail**. Blur is weak. Edge kernels stay sharp. Fast.

A **large kernel (7×7)** looks farther. Blur is **stronger**. You get more context, but **tiny lines can disappear**. Slower, more multiply-adds.

Same idea in a CNN: a larger (or stacked) field of view sees a bigger pattern.

From the notes: *Larger kernel (7×7 vs 3×3) looks at a wider neighborhood: stronger blur, slower, more context, less tiny detail.*

---

10. Explain how convolution is related to feature extraction in CNNs.

Convolution is a sliding-window operation. A kernel moves across the image, computes a weighted sum of local pixels, and produces a **feature map**.

In classical CV we write the kernel by hand (Sobel = edges).

In a **CNN the same slide happens**, but kernel numbers are **learned**. Early layers learn simple things: edges, blobs. Deeper layers learn object parts, then whole objects.

So convolution **is** how a CNN extracts features. Fully connected layers later **decide** the class. Remember: *Conv extracts, Dense decides.*

---

11. How can histograms be used to detect underexposed or overexposed images?

Look at where the bars sit:

- Most pixels piled on the **left** (near 0) → **underexposed** — too dark, shadows have no detail.
- Most pixels piled on the **right** (near 255) → **overexposed** — too bright, highlights are burnt.
- All bars in the **middle**, both ends empty → **low contrast**, not necessarily under/over, but washed.

A healthy photo **spreads** across 0–255. This is the same reading as THEORY-NOTES section 8.

---

12. Discuss the trade-offs between using small vs large structuring elements in morphological operations.

The structuring element is the small shape that slides and edits the object.

Small element: gentle. Thin letters and thin cracks **survive**. Weak on **big** noise. You may need to repeat the operation.

Large element: strong clean-up. Big specks go away, big holes fill. But it can **eat thin objects**, **round corners**, or **merge** two nearby objects into one.

Trade-off: pick size to match the **noise size** and the **object thickness**. For a scanned page, start small.

---

13. How can contour features (area, perimeter, centroid) be used for object classification?

From a contour you can compute numbers:

- **Area** — how many pixels inside. Big coin vs small speck.

- **Perimeter** — length of the outline. A long thin crack has a long perimeter and a small area. A circle is compact.
- **Centroid** — the center point. Used to say *where* the object is, or to track it in the next frame.

Example: keep blobs with area above a cutoff (drop dust). Treat high-area + round perimeter as a coin. Treat low-area + long perimeter as a scratch.

Paper line: *Area, perimeter, centroid — numbers you can use to classify "big vs small", "round vs long".*

---

14. Propose a method to detect cracks in industrial materials using convolution and morphological operations.

Industrial inspection is listed in the notes as a morphology application: detect defects.

1. Convert the photo to **grayscale**.
2. Use **convolution** with a **sharpen** or **edge** kernel (Sobel / Canny) so the crack becomes a bright line. Convolution is extracting the "thin line" feature.
3. **Threshold** that edge map so the crack is white.
4. **Closing** connects broken pieces of the same crack. **Opening** drops specks that are not cracks.
5. **Find contours**. Keep shapes that are **long and thin** (high perimeter, low area). Those are cracks. Draw a box or color the contour.

Small structuring element: a large one would wipe a hairline crack.

---

15. Explain how combining HSV segmentation with contour analysis can improve object detection accuracy.

HSV segmentation gives a **clean color mask**. Only pixels in the right Hue (and useful Saturation/Value) stay white. Wrong-color background is gone. Light change hurts less than RGB, because Hue is stable.

That mask can still have specks and holes, or two objects of the same color.

Contour analysis on the cleaned mask gives the **outline**. Then you can:

- drop tiny contours (noise)
- take **area / perimeter** to reject wrong shapes
- draw a **bounding box** to localise
- use Douglas–Peucker if you need the shape class

Together: color says "this looks like the object's color". Shape says "this blob is the right size and outline". That is more accurate than color alone (which keeps any yellow blob) or shape alone

(which keeps any similar outline in the wrong color).

After this file: close it. Write 5 easy answers and 2 medium answers on paper. Check the wording against THEORY-NOTES, not against a one-line summary.